

HTTP over Reliable UDP (RUDP)

1. Design and Implementation Description

The objective of this project was to design and implement a reliable transport system over a connectionless UDP socket, and build an HTTP/1.0 application layer on top of it. The architecture is divided into three distinct layers:

- **Data Structure (Packet.py):** The RUDPPacket class encapsulates the data payload along with a reliable header containing sequence numbers, acknowledgment numbers, and connection flags (SYN, SYNACK, ACK, FIN, DATA). An MD5 checksum is calculated over all header fields and the payload before transmission to ensure integrity. The class also exposes `simulate_corruption()` to intentionally invalidate the checksum for testing, and `is_corrupt()` to detect tampered or damaged packets on arrival. Packets are serialized to JSON bytes for transmission.
- **Transport Layer (rudp_socket.py):** The ReliableSocket class wraps a standard Python UDP socket and implements a Stop-and-Wait ARQ protocol. The sender transmits one packet at a time and waits for an ACK; if none arrives within the timeout, it retransmits up to `MAX_RETRIES` times before raising a `ConnectionError`. Connection establishment uses a 3-way handshake (SYN → SYNACK → ACK). Teardown uses FIN flags, with the server waiting for the client's FIN and ACKing it, since HTTP/1.0 is always client-closes-first. Duplicate packets are detected via sequence numbers and re-ACKed without being delivered twice. The class also supports configurable packet loss and corruption simulation rates via `set_simulation_rates()` for testing purposes.
- **Application Layer (http_server.py & http_client.py):** The server uses daemon threads to handle multiple clients concurrently without blocking the accept loop. It parses HTTP/1.0 requests supporting GET and POST methods, resolves file paths securely against path-traversal attacks, and responds with appropriate status codes (200 OK, 400 Bad Request, 403 Forbidden, 404 Not Found, 405 Method Not Allowed) and MIME types.

2. Assumptions

- **Network Delays:** Network delays are assumed to be reasonably bounded, making a fixed socket timeout of 2.0 seconds sufficient to detect packet loss without causing excessive retransmissions under normal conditions.
- **Data Integrity:** MD5 is assumed to provide sufficient protection against accidental bit-flips and corruption during transmission. It is not used as a cryptographic security measure.
- **Stateless Connections:** The HTTP connection is non-persistent. Following HTTP/1.0 standard practice, the `Connection: close` header signals that the transport socket is torn down after each single request-response cycle.
- **Single Message In Flight:** Each connection handles one message at a time per direction, which is consistent with the Stop-and-Wait model.

3. Limitations

- **Performance:** The strict Stop-and-Wait ARQ protocol limits throughput by the network's Round Trip Time (RTT). No sliding window, pipelining, or congestion control is implemented.
 - **Payload Size:** A single transmission is limited by the UDP datagram buffer (65,535 bytes), reduced further by JSON serialization overhead, making it unsuitable for large binary file transfers.
 - **HTTP Standard:** The implementation is strictly HTTP/1.0. It does not support chunked transfer encoding, persistent keep-alive connections, or caching headers available in HTTP/1.1 and later.
 - **Binary Files:** File serving reads content as UTF-8 text. Binary files (images, PDFs) are not supported without modification to the file-reading logic.
-

```
D:\Downloadss\CCE Term 8\Computer Networks\Labs\HTTP-over-RUDP>python http_client.py GET /index.html
[CLIENT] Connecting to 127.0.0.1:8080...
[RUDP] Initiating 3-way handshake...
[RUDP] Connection established.
[CLIENT] → GET /index.html
[CLIENT] Waiting for response...

[CLIENT] — Response —————
Status : HTTP/1.0 200 OK
Content-Type: text/html
Content-Length: 66
Connection: close

<h1>Hello from Reliable UDP Server!</h1><p>The lab is working.</p>
[CLIENT] —————
[RUDP] Initiating teardown (FIN)...
[RUDP] FIN attempt 1/10: no response.
[RUDP] FIN attempt 2/10: no response.
[RUDP] Teardown acknowledged.
[RUDP] Socket closed.
```

```
[RUDP] Received SYN from ('127.0.0.1', 57855). Sending SYNACK...
[RUDP] Connection accepted from ('127.0.0.1', 57855).

[SERVER] Handling client ('127.0.0.1', 57855)
[RUDP] Listening for incoming connections...
[SERVER] ← ('127.0.0.1', 57855)
GET /index.html HTTP/1.0
[RUDP] Waiting for client FIN...
[RUDP] Client FIN acknowledged.
[RUDP] Connection closed. Server socket ready for next client.
```

```

D:\Downloadss\CCE Term 8\Computer Networks\Labs\HTTP-over-RUDP>python http_client.py POST /submit "name=Abdullah&project=RUDP"
[CLIENT] Connecting to 127.0.0.1:8080...
[RUDP] Initiating 3-way handshake...
[RUDP] Connection established.
[CLIENT] → POST /submit
[CLIENT] Waiting for response...

[CLIENT] — Response —————
Status : HTTP/1.0 200 OK
Content-Type: text/plain
Content-Length: 25
Connection: close

POST received (26 bytes).
[CLIENT] —————
[RUDP] Initiating teardown (FIN)...
[RUDP] Teardown acknowledged.
[RUDP] Socket closed.

```

```

[RUDP] Received SYN from ('127.0.0.1', 60787). Sending SYNACK...
[RUDP] Connection accepted from ('127.0.0.1', 60787).

[SERVER] Handling client ('127.0.0.1', 60787)
[RUDP] Listening for incoming connections...
[SERVER] ← ('127.0.0.1', 60787)
POST /submit HTTP/1.0
[SERVER] POST body: name=Abdullah&project=RUDP
[RUDP] Waiting for client FIN...
[RUDP] Client FIN acknowledged.
[RUDP] Connection closed. Server socket ready for next client.

```

```

D:\Downloadss\CCE Term 8\Computer Networks\Labs\HTTP-over-RUDP>python -m pytest test_rudp.py -v
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Program Files\Python314\python.exe
cachedir: .pytest_cache
rootdir: D:\Downloadss\CCE Term 8\Computer Networks\Labs\HTTP-over-RUDP
plugins: anyio-4.11.0
collected 12 items

test_rudp.py::TestChecksum::test_checksum_detects_tampered_packet PASSED [ 8%]
test_rudp.py::TestSimulateCorruption::test_simulate_corruption_makes_packet_corrupt PASSED [ 16%]
test_rudp.py::TestPacketLoss::test_loss_rate_1_never_sends PASSED [ 25%]
test_rudp.py::TestSequenceNumber::test_duplicate_packet_not_delivered_twice PASSED [ 33%]
test_rudp.py::TestHandshake::test_three_way_handshake_completes PASSED [ 41%]
test_rudp.py::TestRetransmission::test_data_delivered_after_one_dropped_packet PASSED [ 50%]
test_rudp.py::TestRetransmission::test_retransmits_then_raises_on_total_loss PASSED [ 58%]
test_rudp.py::TestFINteardown::test_fin_closes_connection_cleanly PASSED [ 66%]
test_rudp.py::TestHTTPGet200::test_get_returns_200_ok PASSED [ 75%]
test_rudp.py::TestHTTPGet404::test_get_missing_file_returns_404 PASSED [ 83%]
test_rudp.py::TestHTTPPost::test_post_returns_200 PASSED [ 91%]
test_rudp.py::TestCorruptionEndToEnd::test_corrupted_packet_retransmitted_and_received PASSED [100%]

===== 12 passed in 3.69s =====

```

Test 1 — Checksum Detection (TestChecksum) Verifies that mutating a packet's data field after creation causes `is_corrupt()` to return `True`, confirming the checksum covers the full packet content.

Test 2 — Corruption Simulation (TestSimulateCorruption) Verifies that calling `simulate_corruption()` on a valid packet intentionally invalidates its checksum, making `is_corrupt()` return `True` as required for fault-injection testing.

Test 3 — Packet Loss Simulation (TestPacketLoss) Verifies that setting `loss_rate=1.0` causes `_send_raw()` to never call the underlying `sendto`, confirming the packet loss simulation method works correctly.

Test 4 — Sequence Numbers / Duplicate Packets (TestSequenceNumber) Sends the same DATA packet twice with the same sequence number and verifies that `recv()` delivers the payload exactly once, confirming duplicate detection and re-ACKing without double delivery.

Test 5 — 3-Way Handshake (TestHandshake) Verifies that a client can successfully complete the `SYN → SYNACK → ACK` handshake with a listening server without raising any exception, confirming connection establishment works end-to-end.

Test 6 — Retransmission on Timeout (TestRetransmission — first) Sets `loss_rate=1.0` so every packet is dropped and verifies that `_send_reliable()` raises `ConnectionError` after exhausting all retries, confirming the timeout and retransmission limit behaves correctly.

Test 7 — Successful Delivery After One Drop (TestRetransmission — second) Intercepts `_send_raw` to silently drop the first DATA packet, then verifies the server still receives the correct payload on the retransmission, confirming the ARQ retry mechanism recovers from a single packet loss.

Test 8 — FIN Teardown (TestFINTeardown) Verifies that after a complete request-response cycle, the client's `close()` sends a FIN that the server ACKs cleanly, with no exceptions raised on either side.

Test 9 — HTTP GET 200 OK (TestHTTPGet200) Sends a GET request for an existing file and verifies the response contains the 200 OK status line and the correct file content, confirming the GET method and success status work correctly.

Test 10 — HTTP GET 404 Not Found (TestHTTPGet404) Sends a GET request for a non-existent file and verifies the response contains the 404 status code, confirming the server correctly handles missing resources.

Test 11 — HTTP POST 200 OK (TestHTTPPost) Sends a POST request with a form body and verifies the response contains 200 OK and echoes the correct byte count, confirming the POST method is parsed and handled correctly.

Test 12 — End-to-End Corruption Recovery (TestCorruptionEndToEnd) Intercepts the first DATA packet and manually corrupts its checksum before sending, then verifies the server receives the correct payload on the retransmission, confirming the full corruption-detection and recovery cycle works end-to-end.

No.	Time	Source	Destination	Protocol	Length	Info
[	109	46.674806300	127.0.0.1	UDP	136	53440 → 8080 Len=104
	110	46.675407300	127.0.0.1	UDP	139	8080 → 53440 Len=107
	111	46.675560000	127.0.0.1	UDP	136	53440 → 8080 Len=104
	112	46.676012700	127.0.0.1	UDP	240	53440 → 8080 Len=208
	113	46.677092900	127.0.0.1	UDP	136	8080 → 53440 Len=104
	114	46.678392500	127.0.0.1	UDP	296	8080 → 53440 Len=264
	115	46.678645600	127.0.0.1	UDP	136	53440 → 8080 Len=104
]	116	46.679332900	127.0.0.1	UDP	136	53440 → 8080 Len=104
	117	46.679465500	127.0.0.1	UDP	136	8080 → 53440 Len=104

wireshark Adapter for loopback traffic capture9FUWN3.pcapng